# AI-Based Syntax Error Correction Tool

Compiler Design Project

Concepts Covered: Syntax Analysis • Error Handling

Features: Detect syntax errors • Suggest corrections

**K L University**

Team / Roll Numbers: 2420030563
2420030540
2420030639

Guide:- Uma Shankari

# 1. Introduction

Motivation and project context

## Compiler & Syntax

A compiler translates source code into a target form.

Syntax analysis checks whether tokens follow the grammar of the programming language.

## The Challenge

Small mistakes such as missing brackets, semicolons, or incorrect keywords can stop compilation.

Beginners may find compiler messages difficult to interpret.

## Our Solution

An AI-assisted tool detects syntax errors, identifies likely causes, and suggests a corrected version with a clear explanation.

# 2. Problem Statement

Why an AI-based syntax correction tool is needed

Traditional compiler messages often identify where parsing failed, but may not explain the intended correction.

Multiple syntax errors can produce cascading errors, making the first useful fix hard to identify.

Students and novice programmers spend significant time locating small punctuation or grammar mistakes.

The project aims to automatically detect syntax errors and generate ranked, context-aware correction suggestions.

# 3. Objectives

Project goals

### Detect

Identify syntax violations and their exact source locations.

### Diagnose

Analyze nearby tokens and grammar context to determine the likely cause.

### Suggest

Generate and rank plausible corrections for the detected error.

### Explain

Show what changed and why the correction is appropriate.

# 4. Existing System vs Proposed System

Comparison

### Existing Approach

- Compiler reports syntax errors
- Mostly rule-based messages
- User manually interprets the message
- Limited correction guidance
- Cascading errors may confuse the user

### Proposed AI-Assisted Tool

- Detects syntax errors
- Uses parsing context and code patterns
- Suggests likely fixes
- Ranks alternative corrections
- Explains the change in simple language

# 6. Compiler Design Concepts Used

Syntax Analysis and Error Handling

### Syntax Analysis

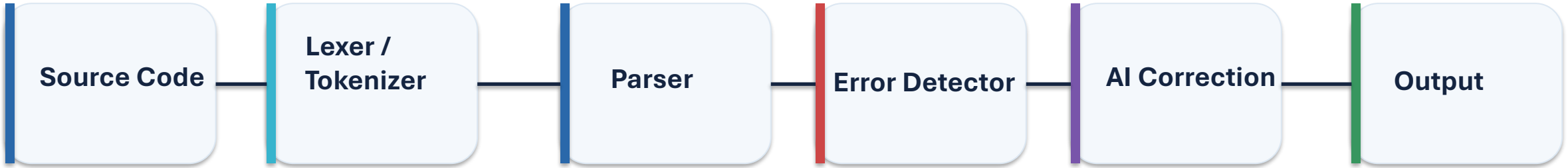
- Tokens are checked against a context-free grammar.
- Parser builds a parse tree / AST.
- Valid programs follow grammar productions.
- Invalid token sequences trigger syntax errors.

### Error Handling

- Detect and localize errors.
- Recover without stopping immediately.
- Avoid misleading cascaded diagnostics.
- Return useful information to the correction engine.

# 7. System Architecture

End-to-end processing pipeline




Token stream

Parse tree / AST

Error location + type

Candidate fixes

Corrected code + explanation

# 8. Algorithm / Working Procedure

Detection → correction → validation

1. Tokenize source code
2. Parse tokens using grammar
3. Capture syntax error and context
4. Generate candidate fixes
5. Score candidates with AI/context
6. Validate corrected code

# 9. Example: Syntax Error Detection

Missing closing parenthesis

### Input Code

```
if (a > b {
    print("a is greater");
}
```

### Suggested Correction

```
if (a > b) {
    print("a is greater");
}
```

### AI Explanation

The condition opens a parenthesis after `if (` but does not close it before `{`.
The tool inserts `)` before the opening brace and re-checks the corrected statement.

# 12. Technologies & Tools

Suggested implementation stack

**Frontend**

HTML / CSS / JavaScript or React

Code editor + error highlighting

**Backend**

Python / Node.js

Parser library or custom grammar

REST API for correction requests

**AI & Validation**

LLM / ML correction Engine

AST or parse-tree Validation

Unit tests + sample programs

# 14. Advantages & Applications

Where the tool is useful

### Advantages

- Faster debugging
- Beginner-friendly explanations
- Fewer repeated compile–fix cycles
- Minimal-edit correction strategy
- Useful for learning compiler concepts

### Applications

- Online coding platforms
- IDE extensions
- Automated code review
- Classroom compiler labs
- Developer productivity tools

# 15. Conclusion & Future Scope

## Conclusion

The proposed AI-Based Syntax Error Correction Tool combines compiler syntax analysis with intelligent correction suggestions.
It detects errors, explains likely causes, proposes fixes, and validates the corrected program.

## Future Scope

- Support multiple programming languages
- Learn from user correction history
- Confidence-aware suggestions
- IDE / VS Code integration
- More advanced semantic error assistance

## Thank You